

KA2 Basic assignment code:

```
#include <Servo.h>

// Pin constants
const int POT_PIN = A0;
const int SERVO_PIN = 9; //has pwm :P

// Calibration Constants for counts and micros value
const int ADC_MIN = 0;
const int ADC_MAX = 1023;
const int PULSE_MIN = 520;
const int PULSE_MAX = 2480;

// freq control
unsigned long lastPulseTime = 0;
int pulseInterval = 1; // Default 20ms (50Hz)

// Mode selection - uncomment the one you want to demonstrate
// #define MODE_CALIBRATED
// #define MODE_GAIN_LARGE
// #define MODE_GAIN_SMALL
#define MODE_FREQ_TEST

// For frequency testing
// Uncomment one frequency at a time for demonstration
//const int PULSE_INTERVAL = 1000; // 1 Hz (1000ms)
// const int PULSE_INTERVAL = 10; // 100 Hz 10ms)
// const int PULSE_INTERVAL = 2; // 500 Hz 2ms)
// const int PULSE_INTERVAL = 1; // 1000 Hz (1ms)

Servo myServo;

void setup() {
  Serial.begin(9600);
  myServo.attach(SERVO_PIN, PULSE_MIN, PULSE_MAX);
} //i,e oin 9 and 600 and 2400ish

void loop() {
  //reminder gain comes from adc in arduino which amplifies smaller movements
  // Read potentiometer
  int sensorValue = analogRead(POT_PIN);

  // Calculate pulse width based on mode
  int pulseWidth;
```

```

#ifdef MODE_CALIBRATED
    pulseWidth = map(sensorValue, ADC_MIN, ADC_MAX, PULSE_MIN, PULSE_MAX);
#elif defined(MODE_GAIN_LARGE)
    // Large gain - amplifies small movements
    int scaledValue = map(sensorValue, ADC_MIN, ADC_MAX, PULSE_MIN - 900,
PULSE_MAX + 900);
    pulseWidth = constrain(scaledValue, 500, 2500); // Safe limits
#elif defined(MODE_GAIN_SMALL)
    // Small gain - reduces range of motion
    pulseWidth = map(sensorValue, ADC_MIN, ADC_MAX, 1350, 1650); // Only 300µs
range
#else // MODE_FREQ_TEST
    pulseWidth = map(sensorValue, ADC_MIN, ADC_MAX, PULSE_MIN, PULSE_MAX);
#endif

// Ensure pulse width is valid (enforce pulse width < period)
pulseWidth = constrain(pulseWidth, 500, min(2500, pulseInterval * 1000 - 1));
//Ensures pulse width stays within safe limits and is less than the pulse
interval

// Send pulse at specified frequency
unsigned long currentTime = millis();
if (currentTime - lastPulseTime >= (unsigned long)pulseInterval) {
    //if statement will checks if enough time has passed since last pulse
    myServo.writeMicroseconds(pulseWidth);
    lastPulseTime = currentTime;

    // Print every time we update
    Serial.print("ADC: ");
    Serial.print(sensorValue);
    Serial.print(" | Pulse: ");
    Serial.print(pulseWidth);
    Serial.print(" µs | Freq: ");
    Serial.print(1000.0 / pulseInterval);
    //sinc pulseInt = ms, freq is 1/s so just convert from ms to s and divide
    Serial.println(" Hz");
}

// Small delay to prevent excessive looping
delay(1);
}

```